

# Computational Engineering

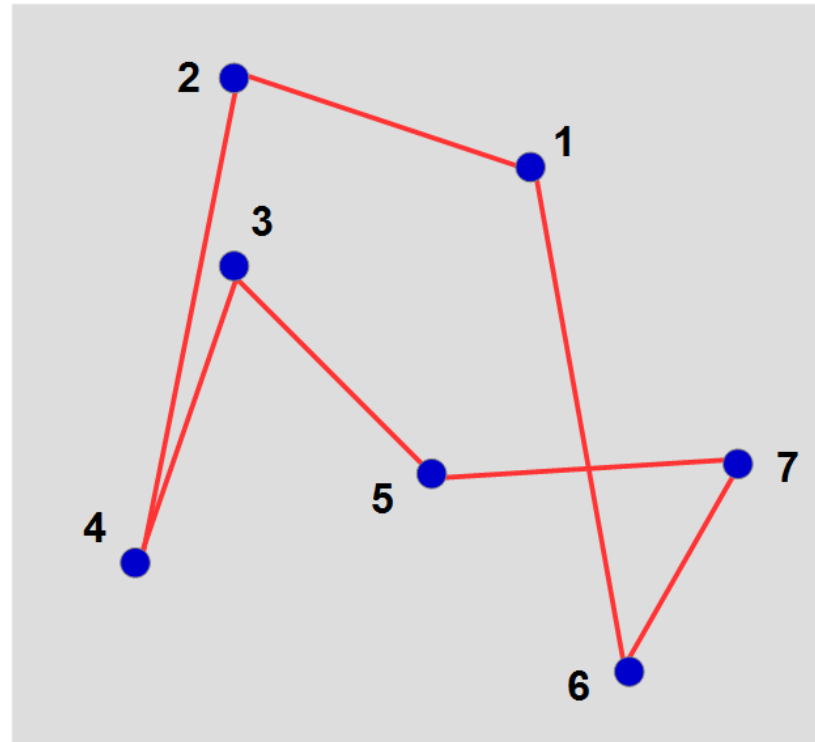
## Stochastische Optimierung / Simulated Annealing

Wie betrachten hier eine spezielle Art von **Optimierungsproblemen**. Ein Optimierungsproblem ist, mathematisch gesprochen, die Suche nach einem Minimum (oder Maximum) einer Funktion  $f = f(\vec{x})$ , evtl. unter einer Reihe von Nebenbedingungen, welchen die Variablen der Funktion unterworfen werden und die idR in Form einer Reihe von Gleichungen  $\varphi_i(\vec{x}) = 0$  formuliert werden können.

Eine spezielle Problemklasse stellen die **kombinatorischen** Optimierungsprobleme dar. Dabei betrachtet man ein **System**, dessen **Zustand**, von der **Reihenfolge** der Anordnung von  $n$  verschiedenen Objekten abhängt. Die Angabe einer speziellen Reihenfolge der  $n$  Objekte stellt dabei einen **Systemzustand** dar.

Als Beispiel wollen wir das berühmte **Problem des Handlungsreisenden** betrachten: Stellen wir uns einen Handlungsreisenden vor, der Kunden an  $n$  verschiedene Orte besuchen muss. Frage: In welcher Reihenfolge soll er die Orte abfahren, damit der dabei zurück gelegte Weg insgesamt **minimal** wird?

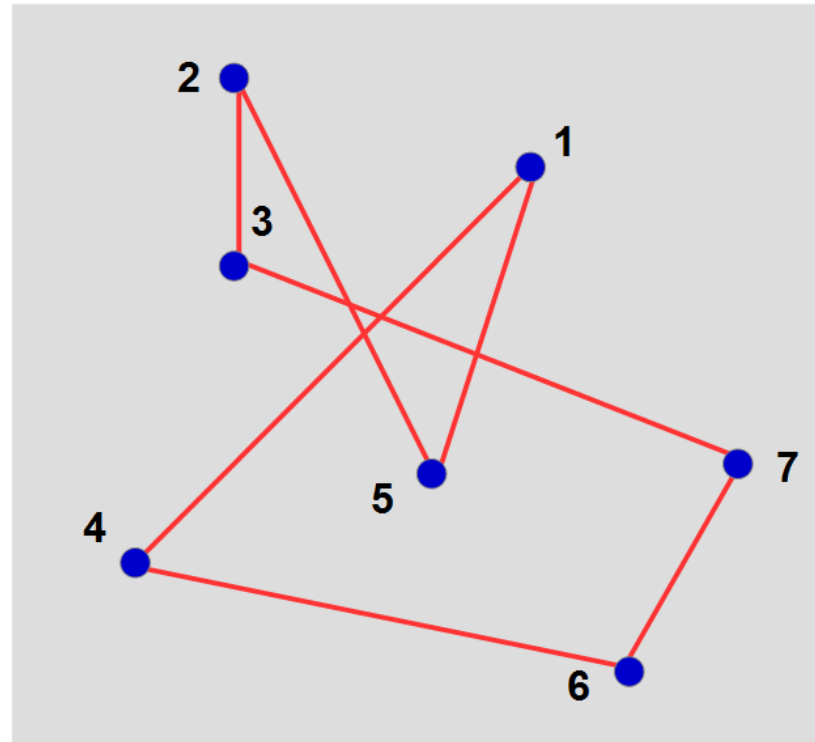
Beispiel: 7 Orte, gekennzeichnet durch jeweils eine feste Zahl (also quasi der Name des Ortes). Eine Mögliche Reiseroute



Systemzustand:

$$s_1 = \{1, 2, 4, 3, 5, 7, 6, 1\}$$

Beispiel: 7 Orte, gekennzeichnet durch jeweils eine feste Zahl (also quasi der Name des Ortes). Eine andere mögliche Reiseroute



Systemzustand:

$$s_2 = \{1, 4, 6, 7, 3, 2, 5, 1\}$$

Der Systemzustand  $s_i$  ist eindeutig durch eine **Permutation** der natürlichen Zahlen  $1, 2, 3, \dots, n$  definiert. Wir nennen  $S$  die Menge aller möglichen Systemzustände

$$S = \{s_i \mid s_i \text{ ist ein möglicher Systemzustand}\}$$

Nun gebe es eine Funktion  $q$ , die jedem Zustand  $s_i$  aus der Menge aller möglichen Zustände  $S$  eine **Bewertung**, oder auch Qualität, in Form einer **einzigsten reellen Zahl** zuweist

$$q : s_i \in S \mapsto \mathbb{R}$$

Man nennt diese Funktion in diesem Zusammenhang oft **Qualitätsfunktion** auch Fitnessfunktion. In unserem Beispiel mit dem Handlungsreisenden, wäre die Funktion  $q$  also die gesamte Weglänge einer ausgewählten Reiseroute  $s_i$ .

Die Frage ist nun also: wie findet man diejenige Reiseroute (Reihenfolge der besuchten Städte), für die die gesamte Weglänge minimal wird? Dies ist ein klassisches kombinatorisches Optimierungsproblem.

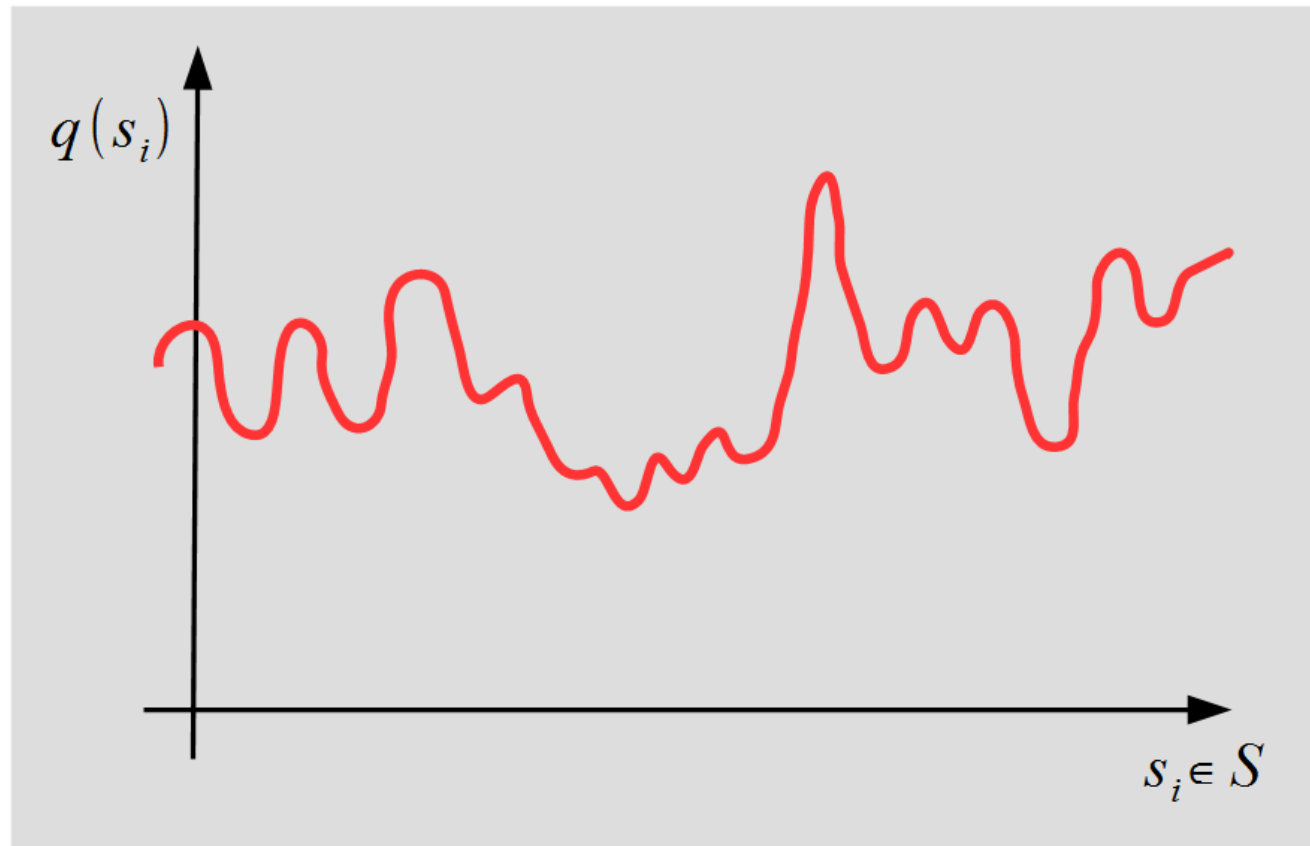
Es gibt  $n!$  Permutationen von  $n$  verschiedenen Objekten, d.h. ohne weiteren einschränkenden Bedingungen, gibt es  $n!$  mögliche Systemzustände. In unserem Beispiel heißt das:  $n!$  verschiedene mögliche Reiserouten.

Die Fakultätsfunktion hat die Eigenschaft, für große Werte von  $n$  **extrem stark** anzuwachsen. Dies hat zur Folge, dass man schon für moderat große Werte von  $n$  zu so vielen verschiedenen Möglichkeiten kommt, dass man selbst mit stärksten Rechnern nicht mehr alle Möglichkeiten (brute force) durchprobieren kann.

Da die "Variable" der Qualitätsfunktion eine Permutation von natürlichen Zahlen ist, kann man die Funktion leider auch nicht im Sinne der Analysis betrachten und mit ihr ein Extremwertproblem konstruieren.

Für diese Art der Problemstellung, welche in der Praxis sehr häufig anzutreffen ist, hat man die Mathematik der **stochastischen Optimierung** entwickelt.

Die typische Qualitäts"funktion" eines kombinatorischen Optimierungsproblems hat i.A. sehr viele, sehr eng beieinander liegende lokale Minima, die sich teils nur sehr wenig im Funktionswert unterscheiden, aber nichts desto trotz nur ein globales Minimum.

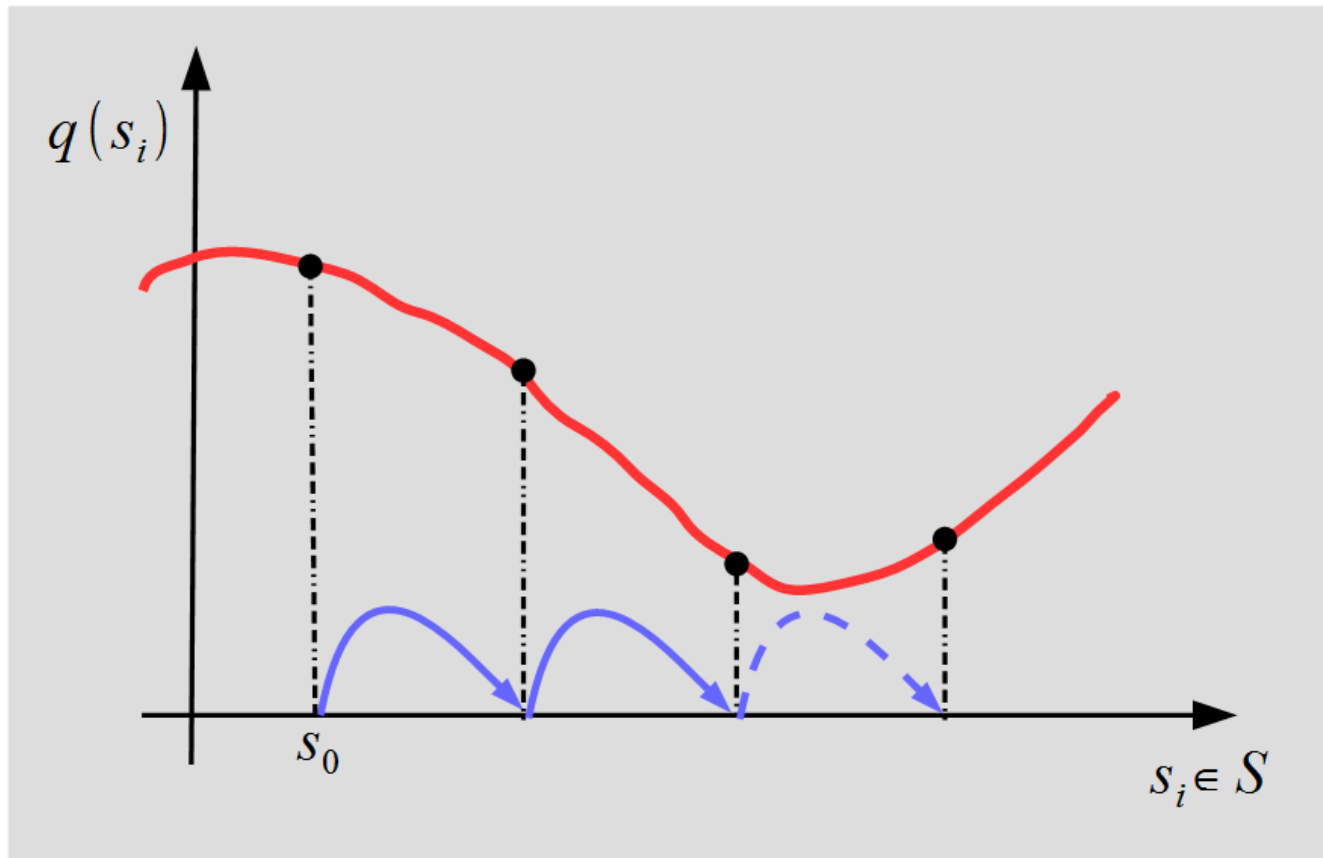


Klassische Optimierungsverfahren benötigen immer eine **Anfangskonfiguration**, die im Falle einer multimodalen Zielfunktion in aller Regel bereits **hinreichend nahe am Optimum** liegen muss, damit das Verfahren konvergiert. Bei der vorliegenden Qualitätsfunktion ist ein solcher Anfangszustand auf Grund der extremen Anzahl der Minima und der nicht stetigen Struktur allerdings unmöglich anzugeben.

Deshalb wählen wir die Startkonfiguration **zufällig** und schließen daran eine Variante des Verfahrens des steilsten Abstieges an:

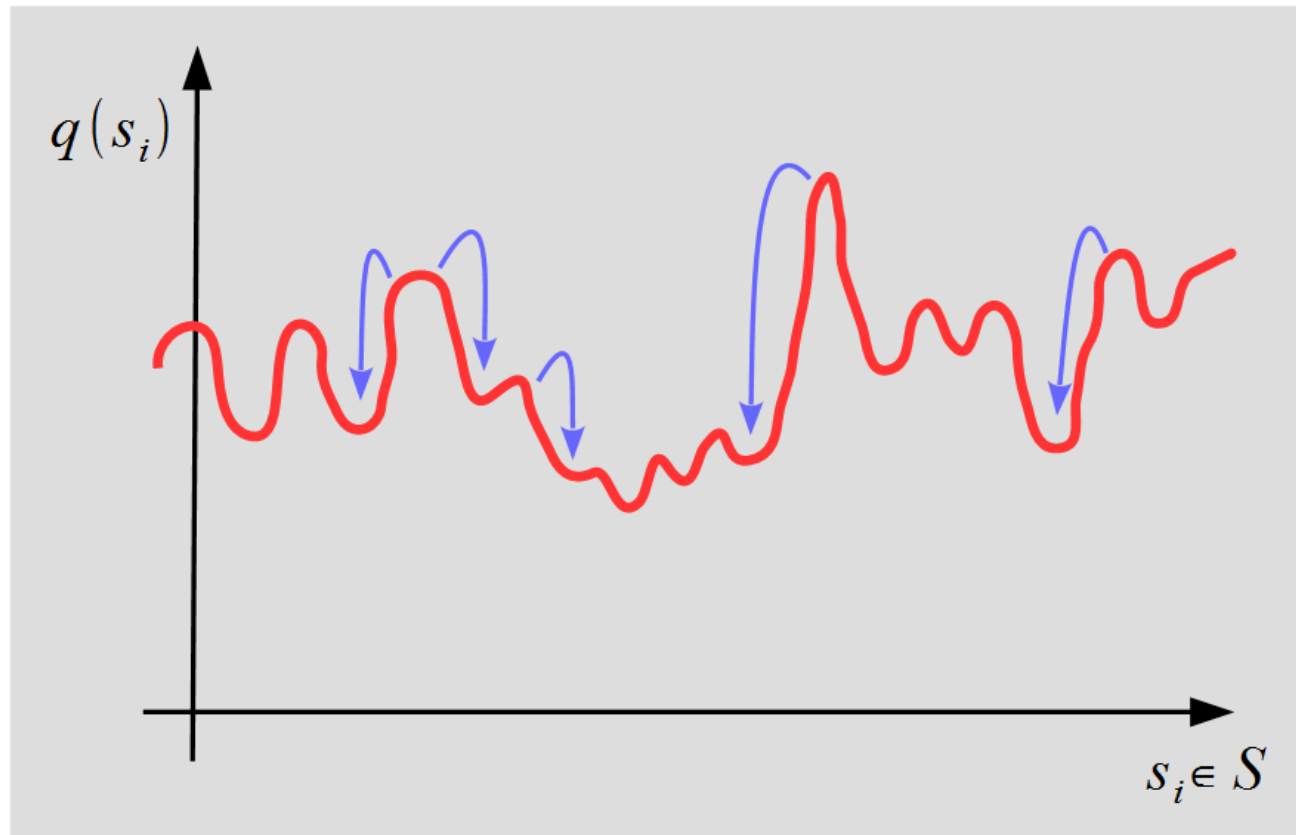
#### Algorithmus: **Lokale Suche**

- $A_0$ : Wähle **zufällige** Startkonfiguration  $s_0 \in S$  und setze  $i = 0$ .
- $A_1$ : Berechne  $q(s_i)$
- $A_2$ : Generiere einen neuen Systemzustand  $s_j$ , durch eine **kleine Veränderung** von  $s_i$  und berechne  $q(s_j)$
- $A_3$ : Wenn  $q(s_j) < q(s_i)$ , dann nehme  $s_j$  als neuen besten Systemzustand und setze  $i = j$ .
- $A_4$ : Gehe zu  $A_2$  und wiederhole so lange bis nach vorgegebener Zahl von erfolglosen Schritten keine Verbesserung mehr eintritt.



Problematisch bei dem Verfahren der lokalen Suche ist, dass durch die Forderung nach der nicht zu großen Veränderung der Zustände  $s_i$  und  $s_j$  (deshalb ja “lokal”), das Verfahren in aller Regel in einem nahe der Startkonfiguration gelegenen Minimum “stecken bleibt”.

Wenn man nun die lokale Suche für **sehr viele** zufällig gewählte Startkonfigurationen durchführt, kann die sich daraus ergebende **Verteilung von Endzuständen** bereits eine genügend große Information über Minimierungsmöglichkeiten des Systems enthalten.



Eine deutliche Verbesserung der lokalen Suche kann erreicht werden, wenn der Minimierungsalgorithmus mit einer **zeitlich abnehmenden Wahrscheinlichkeit** auch eine Verschlechterung Systemzustände akzeptiert. Das führt dazu, dass der Algorithmus nicht im erstbesten lokalen Minimum terminiert. Den sich daraus ergebenden Algorithmus nennt man **Simulated Annealing**, da er erstmals zur Simulation der Abkühlung einer Metallschmelze eingesetzt worden ist.

Die zeitlich (also im Laufe der Iteration) abnehmende Wahrscheinlichkeit auch eine schlechtere Lösung zu akzeptieren wird dabei durch eine (streng) **monoton fallende Funktion**  $T = T(n)$  modelliert, worin  $n$  der Iterationsschritt ist. In anlehnung an des Abkühlungsszenario bezeichnet man diese Funktion in der Regel mit dem Begriff **Temperaturfunktion**.

### Algorithmus: **Simulated Annealing**

$A_0$ : Setze  $n = 0$ ,  $i = 0$ ,  $T_0 = T(0)$  und generiere **zufällige Startkonfiguration**  $s_0$

$A_1$ : Wenn für  $s_i$  eine Abbruchbedingung erfüllt ist, dann stop.

$A_2$ : Erzeuge eine Konfiguration  $s_j$  durch **kleine Veränderung** von  $s_i$ .

$A_3$ : Wenn  $q(s_j) < q(s_i)$  dann setze  $s_i = s_j$  und  $n \rightarrow n + 1$ . Gehe zu  $A_1$ .

$A_4$ : Wenn  $q(s_j) \geq q(s_i)$  dann berechne eine im Intervall  $(0, 1]$  gleichverteilte Zufallszahl  $r$  und

$$p = \exp \left[ -\frac{q(s_j) - q(s_i)}{T(n)} \right]$$

$A_5$ : Wenn  $p > r$  dann setze  $s_i = s_j$ .

$A_6$ : Setze  $n \rightarrow n + 1$  und gehe zu  $A_1$ .

Die Wahl der Kühlfunktion  $T(n)$  ist dabei von entscheidender Bedeutung für die effiziente Konvergenz des Verfahrens. Es kann kein allgemeines Schema angegeben werden, sondern es muss für jeden Fall durch experimentieren heraus gefunden werden, was gut funktioniert. Mögliche Wahl für die Kühlfunktion sind beispielsweise durch die Festlegung

$$T(n+1) = c \cdot T(n), \quad c \in (0, 1)$$

oder

$$T(n+1) = \frac{T(n)}{1 + c \cdot T(n)}, \quad c > 0$$

gegeben. Auch erweist es sich häufig als nützlich, die Temperatur, nach einer bestimmten Zahl von Iterationsdurchläufen ohne Veränderung des Funktionswertes, wieder sprunghaft zu erhöhen, was man als “kicken” bezeichnet.